RELATÓRIO FINAL: HORA-CERTA

1. Identificação

- **Nome do Software:** HoraCerta
- **Curso / Matéria:** Ciência da Computação (2º Período) - Engenharia de Software
- **Instituição:** AFYA – UNIMA
- **Integrantes da Equipe:**
    - Michel Vieira
    - Hugo Braga
    - Pedro Henrique
    - Luan Azevedo
    - Hugo Ferreira

2. Introdução

2.1 Problema

Pequenos estabelecimentos e profissionais autônomos (como barbearias, salões de beleza e consultórios) frequentemente enfrentam desafios operacionais relacionados ao agendamento manual por chat ou agendas de papel. Isso gera problemas como o *no-show* (faltas de clientes), lacunas de horários ociosos, interrupções constantes no atendimento para responder mensagens, além de choques e sobreposições de horários.

2.2 Justificativa e Objetivo

O **HoraCerta** surge como uma solução web automatizada para otimizar a marcação de horários, permitindo que os clientes agendem serviços 24 horas por dia de forma autônoma. O objetivo principal é eliminar o atrito operacional, organizar o fluxo de atendimento e garantir que os prestadores tenham controle total sobre sua grade de horários.

3. Planejamento e Gestão do Projeto

O desenvolvimento seguiu princípios de **Metodologias Ágeis (Scrum)**. Para o gerenciamento de tarefas, a equipe utilizou a ferramenta **Trello**, organizando o fluxo de trabalho em colunas (*A fazer, Em andamento, Em testes, Concluído*). Cada tarefa contou com responsáveis definidos e acompanhamento contínuo do progresso.

- **Link:** https://trello.com/invite/b/6aea050312bd3bc012008ff9/ATTI7500ebf7a52abb946739eb10ee70cd3aE100B881/meu-quadro-do-trello

4. Engenharia de Requisitos

4.1 Principais Stakeholders

- **Cliente / Usuário Final:** Pessoa que busca agendar um serviço de forma rápida e prática pelo celular ou computador.
- **Profissional / Prestador:** Dono ou operador do estabelecimento que gerencia a grade de horários, cadastra serviços e acompanha o painel gerencial.

4.2 Requisitos Funcionais (RF)

- **RF01 - Gestão de Serviços:** O sistema deve permitir o cadastro, edição e exibição de serviços com duração estimada e preço.
- **RF02 - Agendamento Online:** O sistema deve fornecer um formulário público onde o cliente seleciona o profissional, o serviço, a data e o horário disponível.
- **RF03 - Painel Gerencial:** O sistema deve disponibilizar uma interface para o prestador acompanhar a grade de horários ocupados e o status dos atendimentos.
- **RF04 - Trava Anti-Conflito:** O sistema deve bloquear automaticamente horários já reservados para evitar choques de agenda.
- **RF05 - Cancelamento:** O sistema deve permitir o cancelamento da reserva tanto pelo cliente quanto pelo profissional.

4.3 Requisitos Não Funcionais (RNF)

- **RNF01 (Design):** O sistema deve apresentar uma interface limpa, intuitiva e responsiva (abordagem *Mobile-First*), adaptada para computadores e dispositivos móveis.

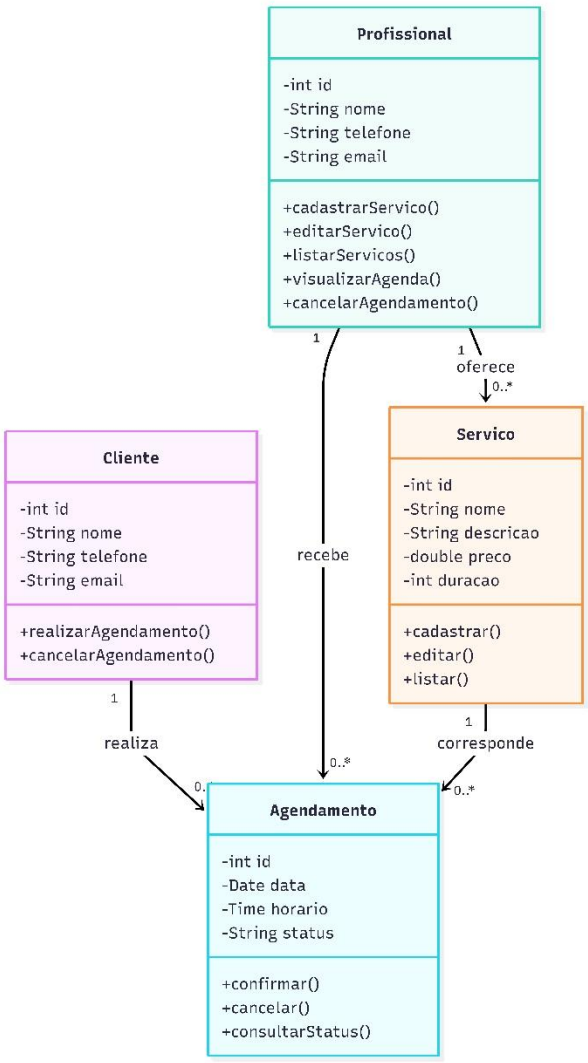
- **RNF02 (Desempenho):** O sistema deve responder às requisições da API de forma ágil e carregar as listagens e disponibilidades de horários sem atrasos perceptíveis.

5. Modelagem UML

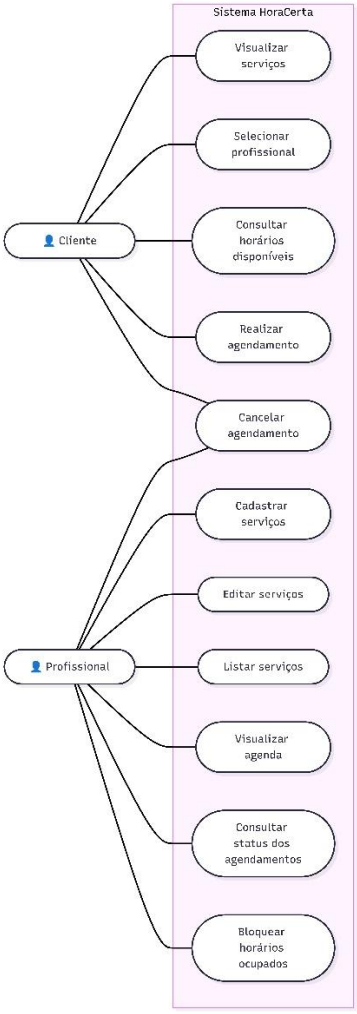
A modelagem do sistema foi estruturada para refletir fielmente a arquitetura desenvolvida:

Ferramenta utilizada: Mermaid

- **Diagrama de Classes:** Detalha as entidades estruturais do sistema (Cliente, Profissional, Serviço e Agendamento), seus atributos primitivos e métodos operacionais essenciais.

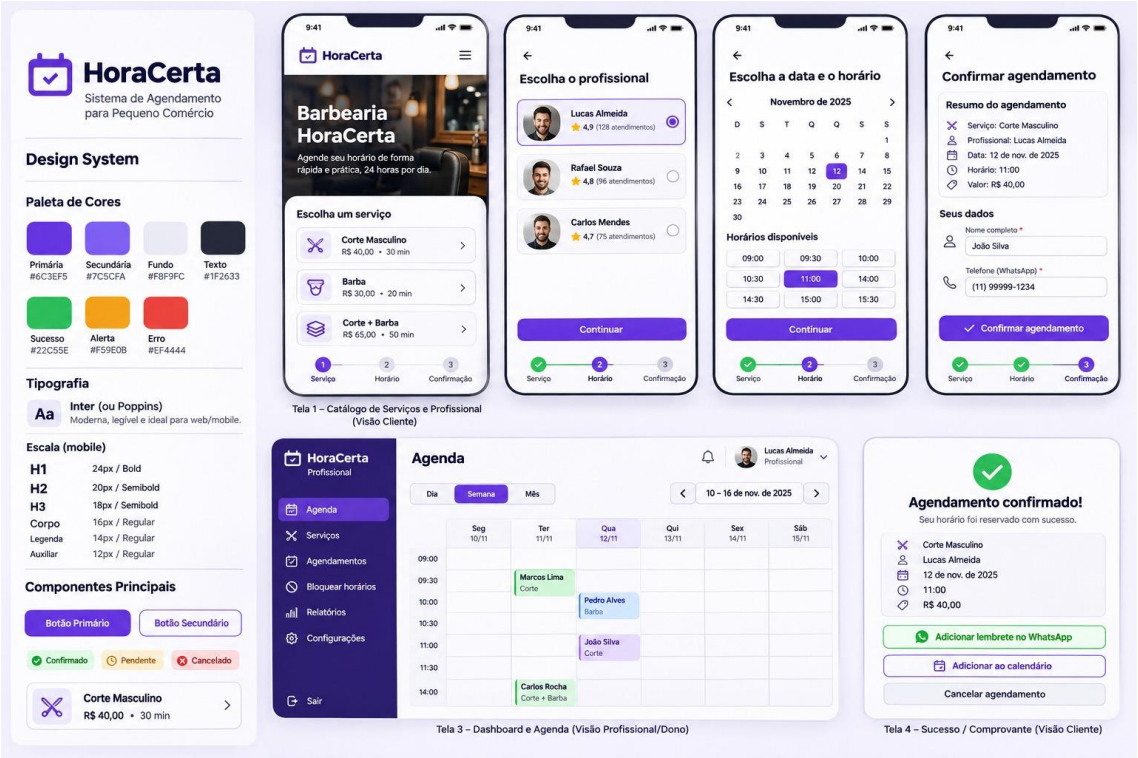


- **Diagrama de Casos de Uso:** Mapeia as interações dos atores (*Cliente* e *Profissional*) com as funcionalidades do sistema (visualizar serviços, selecionar profissional, consultar horários, realizar agendamento, cancelar, cadastrar serviços, bloquear horários).



6. Prototipação do Software

Antes da implementação em código, foram concebidos protótipos de alta fidelidade mapeando as telas principais do sistema:



- **Tela 1:** Catálogo de Serviços e Seleção de Profissional (Visão Cliente).
- **Tela 2:** Seleção de Data, Horários Disponíveis e Confirmação de Agendamento.
- **Tela 3:** Dashboard e Grade de Agenda (Visão Profissional).
- **Tela 4:** Tela de Sucesso e Comprovante de Agendamento com opções de integração.

7. Desenvolvimento do Software

7.1 Tecnologias Utilizadas

- **Frontend:** HTML5, CSS3 / Tailwind, JavaScript (Mobile-First).
- **Backend:** Node.js, TypeScript, Express, Zod (Validação de Dados).
- **Banco de Dados:** SQLite / PostgreSQL com Prisma ORM e Scripts SQL (DDL e DML).

7.2 Funcionalidades Operacionais Implementadas

O sistema encontra-se totalmente funcional com separação entre API Backend e Interface Frontend:

1. **Catálogo Dinâmico e Gestão de Serviços**
2. **Motor de Agendamento Online com Verificação de Conflitos**
3. **Painel Administrativo com Grade de Horários e Relatórios**

8. Testes e Validação

A validação funcional e de integração foi executada cobrindo os seguintes casos de teste:

- **CT01 — Seleção e rolagem dos serviços (visão do cliente):**
  - *Procedimento:* Abrir a visão do cliente com quatro serviços; testar rolagem em viewport móvel.
  - *Resultado Esperado:* Exibir três serviços por vez e permitir alcançar os demais via rolagem.
  - *Resultado Obtido:* Três visíveis inicialmente e o quarto acessível após rolar.
  - *Situação:* **APROVADO**.

- **CT02 — Histórico de agendamentos e filtros de Relatórios:**
  - *Procedimento:* Abrir histórico com registros, rolar e filtrar por status de cancelamento.
  - *Resultado Esperado:* Paginação correta por bloco e filtragem precisa sem corromper totais gerais.
  - *Resultado Obtido:* Listagem correta e filtros aplicados com sucesso.
  - *Situação:* **APROVADO**.

- **CT03 — Rolagem de Serviços e Bloqueio de horários no painel:**
  - *Procedimento:* Testar comportamento das listagens curtas e longas no painel gerencial.
  - *Resultado Esperado:* Sem limitação desnecessária em listas curtas; rolagem correta em listas extensas.
  - *Resultado Obtido:* Comportamento adequado validado sem perdas de dados.
  - *Situação:* **APROVADO**.

- **CT04 — Integração com a API e consulta de disponibilidade:**
    - *Procedimento:* Consultar por requisições GET os endpoints de configuração, serviços, profissionais, relatórios, bloqueios e disponibilidade.
    - *Resultado Esperado:* Respostas HTTP 200 com estruturas JSON válidas.
    - *Resultado Obtido:* Todos os endpoints retornaram status HTTP 200 com dados consistentes.
    - *Situação:* **APROVADO**.

- **CT05 — Compilação do backend e validação do esquema:**
    - *Procedimento:* Executar a compilação TypeScript (npm run build) e a validação do Prisma (npx prisma validate).
    - *Resultado Esperado:* Build sem erros e esquema de banco íntegro.
    - *Resultado Obtido:* Build concluído com sucesso (código de saída 0) e schema validado.

- ***Resumo Geral:*** *5 casos executados, 5 aprovados, 0 reprovados.*

9. Considerações Finais

O desenvolvimento do projeto permitiu à equipe aprender na prática todas as etapas do desenvolvimento de um software. Entre as principais dificuldades superadas destacam-se a **integração síncrona entre o Frontend e o Backend** (lidando com servidores independentes) e o alinhamento do planejamento ágil com a entrega de código funcional.

Como melhorias futuras, a equipe planeja adicionar notificações automatizadas via WhatsApp/E-mail e suporte a múltiplos estabelecimentos na mesma plataforma.

10. Repositório

- **Link do Repositório GitHub:** https://github.com/LuanZedo/Sistema_de_Agendamento